

Tab 1

**Implementasi Perhitungan Parametris pada
Program Python**



Dosen Pengampu:

Fera Putri Ayu Lestari, S.Kom.

Disusun Oleh:

Fa'al Ithaf Nida Nibras	202410370110279
Farrel Maulana Anshari	202410370110287
Kahlil Abrar	202410370110293
Dikha Krisnanda Hafizd	202410370110312

FAKULTAS TEKNIK

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MUHAMMADIYAH MALANG

2026

DAFTAR ISI

BAB 1	
PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Tujuan.....	5
BAB 2	
LANDASAN TEORI/STUDI LITERATUR.....	6
2.1 Konsep Representasi Parametrik.....	6
2.2 Karakteristik Geometri Irisan Kerucut (Conic Sections).....	7
Tabel 2.1 Perbandingan Representasi Parametrik Kurva Konik.....	7
2.2.1 Lingkaran (Circle).....	8
2.2.2 Elips (Ellipse).....	8
2.2.3 Parabola.....	9
2.2.4 Hiperbola (Hyperbola).....	10
2.2.5 Peran Konstanta a dan b serta Parameter Independen (θ dan t).....	10
2.2.6 Studi Literatur.....	11
BAB 3	
IMPLEMENTASI PROGRAM.....	12
3.1 Lingkungan Pengembangan (Environment).....	12
Tabel 3.1 Komponen dan Spesifikasi.....	12
3.2 Potongan Kode Program Utama (Source Code Snippet).....	13
3.2.1 Lingkaran.....	13
Gambar 3.1 Kode Lingkaran.....	13
3.2.2 Elips.....	14
Gambar 3.2 Kode Elips.....	14
3.2.3 Parabola.....	15
Gambar 3.3 Kode Parabola.....	15
3.2.4 Hiperbola.....	16
Gambar 3.4 Kode Hiperbola.....	16
3.3 Desain Antarmuka Interaktif.....	17
Gambar 3.5 Input Keyboard.....	17
Gambar 3.6 Input Slider.....	18
BAB 4	
PEMBAHASAN.....	20
4.1 Analisis Perbandingan (Eksplorasi Variasi Nilai Delta (Step)).....	20
4.1.1 Variasi Step pada Kurva Lingkaran.....	20
Tabel 4.1 Hasil Eksplorasi Variasi Step pada Kurva Lingkaran.....	20
Gambar 4.1 Lingkaran dengan Step 5° dan 10°	21
Gambar 4.2 Lingkaran dengan Step 30° dan 45°	22
4.1.2 Variasi Step pada Kurva Elips.....	22
Tabel 4.2 Hasil Eksplorasi Variasi Step pada Kurva Elips.....	22
Gambar 4.3 Elips dengan Step 1° dan 5°	23
Gambar 4.4 Elips dengan Step 10° dan 30°	23

Gambar 4.5 Elips dengan Step 45°	24
4.1.3 Variasi Step pada Kurva Parabola.....	24
Tabel 4.3 Hasil Eksplorasi Variasi Step pada Kurva Parabola ($X_p = Y_p = 0$, $a = 1$, t : -10 s.d 10).....	24
Gambar 4.6 Parabola dengan Step (t) 0,1 dan 0,5.....	25
Gambar 4.7 Parabola dengan Step (t) 2,0 dan 1,0.....	25
Gambar 4.8 Parabola dengan Step (t) 5,0.....	26
4.1.4 Variasi Step pada Kurva Hiperbola.....	26
Gambar 4.9 Hiperbola dengan Step 1° dan 5°	27
Gambar 4.10 Hiperbola dengan Step 10° dan 30°	27
4.2 Analisis Karakteristik Kurva Berdasarkan Teori Geometri.....	27
4.2.1 Mengapa Kurva menjadi poligon saat step diperbesar?.....	28
4.2.2 Analisis Karakteristik per jenis Kurva.....	28
Tabel 4.5 Perbandingan Panjang Chord pada Hiperbola ($a=80$, $b=60$, $step=10^\circ$).....	29
4.3 Analisis Trade-off antara Kualitas Visual dan Performa Komputasi.....	29
4.3.1 Trade-off pada Kurva Lingkaran.....	29
Tabel 4.6 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Lingkaran.....	30
4.3.2 Trade-off pada Kurva Elips.....	31
4.3.3 Trade-off pada Kurva Parabola.....	32
Tabel 4.8 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Parabola.....	33
4.3.4 Trade-off pada Kurva Hiperbola.....	34
Tabel 4.9 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Hiperbola.....	35
4.4 Rangkuman Analisis Perbandingan.....	35
Tabel 4.10 Rekapitulasi Rekomendasi Nilai Step untuk Kurva Parametris. 36	
BAB 5	
KESIMPULAN.....	37
DAFTAR PUSTAKA.....	38
LAMPIRAN.....	39

DAFTAR GAMBAR

Gambar 3.1 Kode Lingkaran.....	15
Gambar 3.2 Kode Elips.....	16
Gambar 3.3 Kode Parabola.....	17
Gambar 3.4 Kode Hiperbola.....	18
Gambar 3.5 Input Keyboard.....	19
Gambar 3.6 Input Slider.....	20
Gambar 4.1 Lingkaran dengan Step 5° dan 10°	23
Gambar 4.2 Lingkaran dengan Step 30° dan 45°	24
Gambar 4.3 Elips dengan Step 1° dan 5°	25
Gambar 4.4 Elips dengan Step 10° dan 30°	25
Gambar 4.5 Elips dengan Step 45°	26
Gambar 4.6 Parabola dengan Step (t) 0,1 dan 0,5.....	27
Gambar 4.7 Parabola dengan Step (t) 2,0 dan 1,0.....	27
Gambar 4.8 Parabola dengan Step (t) 5,0.....	28
Gambar 4.9 Hiperbola dengan Step 1° dan 5°	29
Gambar 4.10 Hiperbola dengan Step 10° dan 30°	29

DAFTAR TABEL

Tabel 2.1 Perbandingan Representasi Parametrik Kurva Konik.....	10
Tabel 3.1 Komponen dan Spesifikasi.....	15
Tabel 4.1 Hasil Eksplorasi Variasi Step pada Kurva Lingkaran.....	23
Tabel 4.2 Hasil Eksplorasi Variasi Step pada Kurva Elips.....	25
Tabel 4.3 Hasil Eksplorasi Variasi Step pada Kurva Parabola ($X_p = Y_p = 0$, $a = 1$, t : -10 s.d 10).....	27
Tabel 4.4 Hasil Eksplorasi Variasi Step pada Kurva Hiperbola.....	29
Tabel 4.5 Perbandingan Panjang Chord pada Hiperbola ($a=80$, $b=60$, $\text{step}=10^\circ$).....	32
Tabel 4.6 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Lingkaran.....	33
Tabel 4.7 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Elips.....	35
Tabel 4.8 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Parabola.....	36
Tabel 4.9 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Hiperbola.....	38
Tabel 4.10 Rekapitulasi Rekomendasi Nilai Step untuk Kurva Parametris.....	39

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Grafika komputer merupakan bagian dari ilmu komputer yang berhubungan dengan proses pembuatan maupun manipulasi gambar. Bisa juga didefinisikan bahwa grafika komputer adalah suatu proses untuk menciptakan sebuah gambar berdasarkan dari deskripsi objek maupun latar belakang yang terkandung dalam gambar tersebut. Grafika komputer merupakan teknik untuk membuat gambar objek sesuai dengan objek tersebut di alam nyata. Pada perkembangan grafika komputer, kemampuan untuk memodelkan objek geometris primitif menjadi pondasi utama dalam berbagai aplikasi mulai dari desain teknik, simulasi ilmiah, pengembangan game hingga visualisasi data ilmiah (Eze et al., 2022). Kemampuan tersebut dapat menjadi dasar penting untuk memahami algoritma pembangkitan kurva dan objek geometris yang efisien, sehingga menghasilkan visualisasi yang akurat serta memiliki performa komputasi yang jelas dan juga bagus.

Salah satu tantangan pada grafika komputer adalah menghadirkan kurva berkelanjutan pada layar komputer yang pada dasarnya bersifat diskrit berbasis piksel (Latour & Droogenbroeck, 2024). Layar komputer hanya bisa menampilkan sejumlah titik dengan resolusi terbatas. Sehingga diperlukan teknik matematika yang tepat untuk mendekati bentuk kurva ideal dari fungsi matematika yang bersifat berkelanjutan tak hingga. Kurva konik (conic sections) yaitu lingkaran, elips, parabola dan hiperbola merupakan bentuk geometris fundamental yang diperoleh dari irisan bidang datar dengan kerucut ganda. Keempat kurva ini memiliki peran krusial dalam berbagai bidang seperti orbit planet dalam fisika mengikuti bentuk elips, parabola digunakan pada antena parabola dan lintasan proyektil. Sedangkan hiperbola diaplikasikan dalam sistem navigasi dan sonar.

Untuk mengimplementasikan kurva-kurva tersebut secara komputasional, representasi parametrik dipilih sebagai pendekatan utama. Berbeda dengan persamaan yang mendefinisikan kurva secara tidak langsung melalui relasi antar variabel. Representasi parametrik secara eksplisit mengekspresikan koordinat (x, y) sebagai fungsi dari sebuah parameter tunggal yang bebas. Pendekatan ini memberikan fleksibilitas tinggi dalam kontrol arah, rentang dan kerapatan titik yang dibangkitkan. Selain itu, representasi

parametrik memudahkan proses rendering karena setiap titik pada kurva dapat dihitung secara berurutan berdasarkan perubahan nilai parameter (Unterguggenberger et al., 2024). Dengan demikian, proses visualisasi menjadi lebih efisien dan memungkinkan analisis terhadap pengaruh variasi parameter, seperti nilai step (Δt) terhadap kualitas bentuk kurva yang dihasilkan.

1.2 Tujuan

1. Mengimplementasikan algoritma pembangkitan kurva seperti (lingkaran, elips, parabola dan hiperbola) menggunakan pendekatan parametrik dengan bahasa pemrograman Python dan library matplotlib.
2. Merancang antarmuka interaktif yang memungkinkan pengguna mengatur parameter kurva secara *real-time* melalui inputan sehingga bisa bereksperimen dan menganalisis outputnya.
3. Menganalisis pengaruh setiap parameter terhadap bentuk, posisi dan ukuran kurva yang dihasilkan dari program python.
4. Menganalisis *trade-off* antara kualitas kurva dan beban komputasi dalam kaitannya dengan nilai step (delta parameter).
5. Memahami konsep bentuk poligon yang terbentuk saat titik-titik dihubungkan dengan garis lurus untuk membentuk kurva pada layar berbasis piksel.

BAB 2

LANDASAN TEORI/STUDI LITERATUR

2.1 Konsep Representasi Parametrik

Dalam grafika komputer, terdapat dua pendekatan utama untuk mempresentasikan kurva Geometris, yakni representasi implisit/kartesian dan representasi parametrik. Representasi implisit mendefinisikan kurva sebagai himpunan titik yang memenuhi persamaan tertentu, misalnya $x^2 + y^2 = r^2$ untuk lingkaran. Pendekatan ini memiliki keterbatasan dalam komputasi grafis karena sulit menentukan urutan titik secara berurutan yang dibutuhkan saat merender kurva ke layar piksel.

Representasi parametrik mendefinisikan koordinat setiap titik pada kurva sebagai fungsi dari satu variabel bebas yang disebut parameter. Program cukup melakukan iterasi nilai parameter secara berurutan untuk menghasilkan seluruh titik penyusun kurva. Pendekatan ini jauh lebih alami untuk implementasi komputasi karena berbentuk perulangan (Loop) sederhana.

Layar komputer bersifat diskrit (berbasis piksel), sehingga kurva berkelanjutan tidak bisa digambar langsung. Representasi parametrik memungkinkan program membangkitkan titik-titik diskrit yang kemudian dihubungkan dengan segmen garis pendek, menciptakan ilusi kurva kontinu. Keunggulan lain representasi parametrik meliputi kemudahan mengontrol kerapatan titik melalui nilai step, fleksibilitas mempresentasikan kurva tertutup maupun terbuka, serta kemudahan melakukan transformasi geometris seperti translasi dan skala.

Perbedaan mendasar antara persamaan implisit dan parametrik dapat dilihat pada contoh lingkaran: secara implisit ditulis $F(x, y) = x^2 + y^2 - r^2 = 0$, sedangkan secara parametrik di tulis $x = r \cdot \cos(\theta)$ dan $y = r \cdot \sin(\theta)$ dengan θ sebagai parameter bebas. Pada representasi parametrik, cukup melakukan θ dari 0° hingga 360° untuk menghasilkan seluruh titik keliling lingkaran.

2.2 Karakteristik Geometri Irisan Kerucut (*Conic Sections*)

Irisan kerucut (conic sections) adalah kurva-kurva yang dihasilkan dari perpotongan sebuah bidang datar dengan kerucut ganda (double cone). Terdapat empat jenis irisan kerucut, yaitu lingkaran, elips, parabola, dan hiperbola, masing-masing terbentuk tergantung pada sudut dan posisi bidang pemotong terhadap kerucut. Keempat kurva ini memiliki karakteristik geometri yang berbeda dan dapat direpresentasikan dengan persamaan parametrik yang berbeda pula.

Representasi parametrik setiap kurva konik menggunakan konstanta geometri yang khas: jari-jari r untuk lingkaran, semi-sumbu a dan b untuk elips dan hiperbola, serta konstanta a dan parameter t untuk parabola. Parameter independen yang digunakan adalah θ (theta) untuk lingkaran, elips, dan hiperbola, sedangkan parabola menggunakan t yang berjalan linear.

Tabel 2.1 Perbandingan Representasi Parametrik Kurva Konik

Kurva	Rumus Parametrik	Parameter Utama	Rentang Parameter
Lingkaran	$x = x_c + r \cdot \cos(\theta), y = y_c + r \cdot \sin(\theta)$	x_c, y_c, r, θ	$\theta \in [0^\circ, 360^\circ]$
Elips	$x = x_c + a \cdot \cos(\theta), y = y_c + b \cdot \sin(\theta)$	x_c, y_c, a, b, θ	$\theta \in [0^\circ, 360^\circ]$
Parabola	$x = x_p + a \cdot t^2$ $, y = y_p + 2 \cdot a \cdot t$	x_p, y_p, a, t	$\theta \in [t_1, t_2]$ (linear)

Hiperbola	$x = xc + a \cdot \sec(\theta), y = yc$ $+ b \cdot \tan(\theta)$	xc, yc, a, b, θ	$\theta \in$ $(-90^\circ, 90^\circ)$ $\cup$ $(90^\circ, 270^\circ)$
-----------	---	------------------------	--

2.2.1 Lingkaran (Circle)

Lingkaran didefinisikan sebagai himpunan titik yang memiliki jarak sama (jari-jari r) terhadap titik pusatnya (xc, yc) . Dalam representasi parametrik, lingkaran dibentuk menggunakan fungsi cosinus dan sinus dengan sudut θ sebagai parameter bebas yang berjalan dari 0° hingga 360° .

Rumus: $x = xc + a \cdot \cos(\theta)$ dan $y = yc + b \cdot \sin(\theta)$

Setiap parameter memiliki peran yang jelas: pasangan (xc, yc) menentukan posisi pusat lingkaran pada bidang koordinat, nilai r menentukan besar jari-jari (ukuran lingkaran), dan θ berperan sebagai sudut yang berputar mengelilingi pusat dari 0° sampai 360° sehingga seluruh keliling terbentuk. Parameter step menentukan interval perubahan θ pada setiap iterasi, yang secara langsung mengontrol kerapatan titik dan kemulusan kurva.

Karena lingkaran hanya memiliki satu parameter pembentuk (r), jarak antar titik yang dihasilkan pada keliling bersifat seragam untuk setiap nilai step yang sama. Hal ini menjadikan lingkaran sebagai kurva yang paling toleran terhadap perubahan nilai step dibandingkan kurva konik lainnya.

2.2.2 Elips (Ellipse)

Elips adalah irisan kerucut yang terbentuk ketika bidang pemotong memiliki sudut lebih kecil dari sudut kerucut terhadap sumbu. Elips ditentukan oleh dua jari-jari: semi-sumbu horizontal a dan semi-sumbu vertikal b , di mana $a \neq b$. Apabila $a = b$, bentuk yang dihasilkan adalah lingkaran dengan $r = a = b$.

Rumus: $x = xc + a \cdot \cos(\theta)$ dan $y = yc + b \cdot \sin(\theta)$

Parameter a (semi-sumbu horizontal) menentukan seberapa jauh elips membentang ke kanan dan kiri dari pusat, dihitung menggunakan cosinus. Parameter b (semi-sumbu vertikal) menentukan seberapa jauh elips membentang ke atas dan bawah, dihitung menggunakan sinus. Titik pusat (xc, yc) berfungsi memindahkan posisi elips tanpa mengubah bentuknya.

Karena $a \neq b$, kecepatan perubahan koordinat tidak seragam di sepanjang kurva: titik-titik lebih rapat di area ujung sumbu pendek dan lebih renggang di area ujung sumbu panjang. Kondisi ini menyebabkan kekasaran visual yang tidak merata ketika nilai step diperbesar, berbeda dengan lingkaran yang distribusi titiknya merata di seluruh sisi.

2.2.3 Parabola

Parabola terbentuk ketika bidang pemotong sejajar dengan salah satu sisi kerucut. Berbeda dengan lingkaran dan elips, parabola merupakan kurva terbuka yang memiliki satu titik ekstrem (vertex/puncak) dan membuka ke satu arah secara tak terhingga.

Rumus: $x = xp + a \cdot t^2$ dan $y = yp + 2 \cdot a \cdot t$

Parameter t adalah variabel bebas yang bertambah linear dari t_1 hingga t_2 , bukan sudut dalam derajat. Parameter (xp, yp) menentukan posisi puncak parabola pada bidang koordinat. Parameter a menentukan arah dan lebar bukaan: jika $a > 0$ parabola membuka ke kanan, jika $a < 0$ membuka ke kiri, dan semakin besar $|a|$ kurva terlihat semakin lebar karena perubahan koordinat menjadi lebih cepat.

Sifat kuadratik pada persamaan $x = xp + a \cdot t^2$ menyebabkan distribusi titik tidak seragam: titik-titik lebih rapat di sekitar puncak (t mendekati nol) dan semakin renggang di ujung rentang ($|t|$ besar). Ini merupakan perbedaan mendasar parabola dibanding lingkaran dan elips yang menggunakan parameter sudut periodik.

2.2.4 Hiperbola (Hyperbola)

Hiperbola terbentuk ketika bidang pemotong memiliki sudut lebih besar dari sudut kerucut terhadap sumbu, sehingga memotong kedua sisi kerucut ganda dan menghasilkan dua cabang kurva yang terpisah. Hiperbola memiliki dua asimtot yang menjadi batas pendekatan kurva namun tidak pernah dicapai.

Rumus: $x = xc + a \cdot \sec(\theta) = xc + a/\cos(\theta)$ dan $y = yc + b \cdot \tan(\theta)$

Parameter a (semi-transversal) menentukan jarak puncak hiperbola dari pusat dan lebar bukaan cabang secara horizontal. Parameter b (semi-konjugat) menentukan kemiringan asimtot dan kecuraman cabang: semakin besar b cabang semakin curam, semakin kecil b cabang semakin landai. Jika $b = 0$, hiperbola berubah menjadi garis lurus horizontal (kurva rusak).

Hiperbola memiliki karakteristik unik dibanding kurva konik lainnya karena fungsi $\sec(\theta)$ dan $\tan(\theta)$ bersifat divergen (menuju tak terhingga) saat θ mendekati 90° dan 270° . Akibatnya jarak antar titik berurutan tidak konstan terhadap step dan jauh lebih ekstrem di area mendekati asimtot dibanding area dekat vertex. Kondisi ini menjadikan hiperbola sebagai kurva yang paling sensitif terhadap perubahan nilai step.

2.2.5 Peran Konstanta a dan b serta Parameter Independen (θ dan t)

Konstanta a dan b pada kurva konik memiliki peran geometri yang spesifik. Pada elips, a adalah panjang semi-sumbu horizontal dan b adalah panjang semi-sumbu vertikal. Pada hiperbola, a menentukan posisi puncak (jarak dari pusat ke vertex) sedangkan b menentukan kemiringan asimtot. Perbandingan b/a pada hiperbola menentukan sudut asimtot terhadap sumbu transversal.

Parameter independen θ pada lingkaran, elips, dan hiperbola berperan sebagai sudut yang berputar mengelilingi pusat kurva. Nilai fungsi trigonometri seperti $\cos(\theta)$ dan $\sin(\theta)$ digunakan untuk menghitung komponen horizontal dan vertikal titik pada lingkaran maupun elips, sedangkan hiperbola menggunakan fungsi $\sec(\theta)$ dan $\tan(\theta)$ untuk merepresentasikan bentuk kurva yang memiliki cabang tak hingga.

Berbeda dengan kurva lainnya, parabola menggunakan parameter independen t , yaitu parameter bebas yang berubah secara linear. Nilai t digunakan untuk menghasilkan koordinat titik melalui persamaan parametrik sehingga membentuk kurva parabola.

Selain itu, nilai step ($\Delta\theta$ atau Δt) menentukan interval perubahan parameter pada setiap iterasi. Semakin kecil nilai step, semakin banyak titik yang dihasilkan sehingga kurva terlihat lebih halus, sedangkan nilai step yang lebih besar menghasilkan jumlah titik lebih sedikit sehingga proses rendering menjadi lebih cepat namun kualitas visual kurva menurun. Penggunaan representasi parametrik memberikan fleksibilitas dalam pembangkitan titik-titik kurva serta mempermudah proses visualisasi dan rendering pada aplikasi grafika komputer (Das et al., 2021; Karakılıç et al., 2025).

2.2.6 Studi Literatur

Representasi parametrik merupakan salah satu metode yang banyak digunakan dalam grafika komputer karena mampu menghasilkan koordinat titik secara efisien dan memberikan fleksibilitas dalam proses pemodelan geometri. Das et al. (2021) pada penelitian *Cloud2 Curve: Generation and Vectorization of Parametric Sketches* menjelaskan bahwa representasi parametrik memungkinkan suatu objek direpresentasikan sebagai kumpulan kurva yang lebih ringkas dan mudah dimanipulasi dibandingkan representasi berbasis piksel. Pendekatan tersebut mempermudah proses rendering, pengeditan bentuk objek, serta menghasilkan kualitas visual yang lebih baik pada aplikasi grafika komputer.

Sementara itu, Karakılıç et al. (2025) dalam penelitian *Bézier Curves and Surfaces with the Blending (α, λ, s) -Bernstein Basis* menunjukkan bahwa perubahan parameter pada kurva parametrik dapat memengaruhi bentuk geometri tanpa mengubah struktur dasar kurva. Penelitian tersebut menegaskan bahwa representasi parametrik memberikan fleksibilitas yang tinggi dalam proses pemodelan, sehingga bentuk kurva dapat dikendalikan melalui perubahan parameter tertentu sesuai kebutuhan aplikasi grafika komputer.

BAB 3

IMPLEMENTASI PROGRAM

3.1 Lingkungan Pengembangan (*Environment*)

Lingkungan pengembangan yang digunakan dalam membangun program visualisasi kurva konik parametrik ini mencakup spesifikasi perangkat keras, sistem operasi bahasa pemograman, serta *library* grafika yang digunakan selama proses implementasi sebagai berikut:

Tabel 3.1 *Komponen dan Spesifikasi*

Komponen	Spesifikasi
Spesifikasi Perangkat Keras minimum	Processor dual-core, RAM 4 GB
Sistem Operasi	Windows 11 (64-bit)
Bahasa Pemograman	Python 3
Library Grafika	Matplotlib

Program dibangun menggunakan paradigma berorientasi objek (*Object-Oriented Programming*), di mana setiap jenis kurva konik (lingkaran, elips, parabola dan hiperbola) memiliki fungsi perhitungan tersendiri yang dipisahkan dari fungsi rendering. Pemisahan ini bertujuan agar proses perhitungan titik koordinat tidak bercampur dengan proses visualisasi, sehingga program lebih mudah dikembangkan dan dipelihara.

Antarmuka program dibangun menggunakan Tkinter yang menyediakan komponen input berupa slider dan keyboard (*entry box*) untuk mengatur nilai parameter, sedangkan visualisasi kurva ditampilkan menggunakan Matplotlib yang diintegrasikan ke dalam jendela Tkinter melalui canvas `FigureCanvasTkAgg`.

3.2 Potongan Kode Program Utama (*Source Code Snippet*)

Program dikembangkan menggunakan kode yang berbeda pada setiap jenis kurva, dengan perbedaan utama terletak pada persamaan matematis yang digunakan, dan setiap kurva memiliki class tersendiri seperti `compute_lingkaran`, `compute_elips`, `compute_parabola`, dan `compute_hiperbola`.

3.2.1 Lingkaran

Kurva lingkaran menerapkan persamaan parametrik sebagai berikut:

$$x = xc + r \cos(\theta) \quad y = yc + r \sin(\theta)$$

Parameter yang digunakan persamaan tersebut terdiri atas koordinat pusat (xc , yc), jari-jari (r), dan step derajat yang berfungsi menentukan jarak antar sudut yang akan dihitung.

```
def compute_lingkaran(self):
    xc = self.get_val("xc")
    yc = self.get_val("yc")
    r = self.get_val("r")
    step = self.get_val("step_derajat")
    sudut_wajib = [0, 90, 180, 270, 360]
    sudut_set = set()
    t = 0.0
    while t <= 360.0 + 1e-9:
        sudut_set.add(round(t, 6))
        t += step
    for s in sudut_wajib:
        sudut_set.add(round(s, 6))
    points = []
    for s in sorted(sudut_set):
        rad = math.radians(s)
        x = bersihkan_error(xc + r * math.cos(rad))
        y = bersihkan_error(yc + r * math.sin(rad))
        points.append((s, x, y))
    return points
```

Gambar 3.1 Kode Lingkaran

Logika perulangan (looping) pada fungsi `compute_lingkaran` dilakukan dengan cara membangkitkan terlebih dahulu daftar sudut (`sudut_set`) menggunakan struktur data set agar tidak terjadi duplikasi nilai. Proses `while t <= 360.0` melakukan iterasi

mulai dari sudut 0° hingga 360° dengan kenaikan sebesar nilai `step` yang dimasukkan oleh pengguna. Setelah daftar sudut terbentuk, setiap sudut dikonversi dari satuan derajat ke radian dengan menggunakan fungsi `math.radians(s)`, mengingat fungsi trigonometri pada bahasa pemrograman python bekerja dalam satuan radian. Selanjutnya, koordinat (x, y) dihitung berdasarkan rumus parametrik lingkaran, kemudian hasilnya dibersihkan dari kemungkinan error pembulatan floating-point melalui fungsi `bersihkan_error()`, dan disimpan ke dalam list `points` untuk diproses pada tahap rendering.

3.2.2 Elips

Kurva elips dibangkitkan dengan menambahkan dua nilai semi-sumbu yang berbeda, yaitu semi-sumbu horizontal (a), dan semi-sumbu vertikal (b), berdasarkan persamaan berikut:

$$x = xc + a \cos(\theta) \qquad y = yc + b \sin(\theta)$$

```
def compute_elips(self):
    xc = self.get_val("xc")
    yc = self.get_val("yc")
    a = self.get_val("a")
    b = self.get_val("b")
    step = self.get_val("step_derajat")
    sudut_wajib = [0, 90, 180, 270, 360]
    sudut_set = set()
    t = 0.0
    while t <= 360.0 + 1e-9:
        sudut_set.add(round(t, 6))
        t += step
    for s in sudut_wajib:
        sudut_set.add(round(s, 6))
    points = []
    for s in sorted(sudut_set):
        rad = math.radians(s)
        x = bersihkan_error(xc + a * math.cos(rad))
        y = bersihkan_error(yc + b * math.sin(rad))
        points.append((s, x, y))
    return points
```

Gambar 3.2 Kode Elips

Pada fungsi `compute_elips` pada dasarnya identik dengan `compute_lingkaran`, yaitu membangkitkan himpunan sudut mulai dari 0° sampai dengan 360° berdasarkan step yang telah ditentukan. Perbedaan utama terletak pada perhitungan koordinat, yaitu komponen x dikalikan dengan semi-sumbu horizontal (a) dan komponen Y dikalikan dengan semi-sumbu vertikal (b). Oleh karena nilai a dan b dapat berbeda, jarak antar titik (chord) yang terbentuk di sepanjang keliling elips menjadi tidak seragam, yaitu lebih rapat pada area sumbu pendek dan lebih renggan pada area sumbu panjang. Dengan demikian, semakin besar nilai step yang digunakan, semakin terlihat bentuk poligon yang tidak proporsional pada hasil rendering elips.

3.2.3 Parabola

Kurva parabola menggunakan parameter bebas t yang berjalan dari t1 sampai dengan t2, berdasarkan persamaan berikut:

$$x = xp + a \cdot t^2 \quad y = yp + 2 \cdot a \cdot t$$

```
def compute_parabola(self):
    xp = self.get_val("xp")
    yp = self.get_val("yp")
    a = self.get_val("a")
    t1 = self.get_val("t1")
    t2 = self.get_val("t2")
    step = self.get_val("step")
    if step <= 0:
        messagebox.showerror("Input Error", "Step harus > 0!")
        raise ValueError("step <= 0")
    t_set = set()
    t = t1
    while t <= t2 + 1e-9:
        t_set.add(round(t, 6))
        t += step
    t_set.add(round(t1, 6))
    t_set.add(round(t2, 6))
    points = []
    for t_val in sorted(t_set):
        x = bersihkan_error(xp + a * t_val ** 2)
        y = bersihkan_error(yp + 2 * a * t_val)
        points.append((t_val, x, y))
    return points
```

Gambar 3.3 Kode Parabola

Berbeda dengan kurva lingkaran dan elips yang parameternya bersifat periodik ($0^\circ - 360^\circ$), parameter t pada parabola bersifat linear dan tidak periodik, sehingga proses perulangan dilakukan mulai dari nilai t_1 hingga t_2 dengan kenaikan sebesar step yang dimasukkan oleh pengguna. Nilai t_1 dan t_2 ditambahkan secara khusus ke dalam himpunan t_set dengan tujuan agar titik awal dan titik akhir kurva tetap tergambar meskipun tidak tepat berada pada kelipatan step . Pada setiap iterasi, koordinat x dihitung secara kuadratik (nilai t dikuadratkan), sedangkan koordinat y dihitung secara linear terhadap t . Akibatnya, perubahan koordinat terlihat lebih drastis pada bagian ujung rentang t dibandingkan dengan bagian tengah mendekati nol.

3.2.4 Hiperbola

Kurva hiperbola dibangkitkan dengan menggunakan fungsi $\sec$ dan $\tan$ yang bersifat divergen pada saat mendekati sudut 90° dan 270° , berdasarkan persamaan berikut:

$$x = x_c + a / \cos(t) \qquad y = y_c + b * \tan(t)$$

```
def compute_hiperbola(self):
    xc = self.get_val("xc")
    yc = self.get_val("yc")
    a = self.get_val("a")
    b = self.get_val("b")
    step = self.get_val("step_derajat")
    sudut_wajib = [0, 90, 180, 270, 360]
    sudut_set = set()
    t = 0.0
    while t <= 360.0 + 1e-9:
        sudut_set.add(round(t, 6))
        t += step
    for s in sudut_wajib:
        sudut_set.add(round(s, 6))
    points = []
    for s in sorted(sudut_set):
        rad = math.radians(s)
        cos_val = math.cos(rad)
        if abs(cos_val) > 1e-10:
            x = bersihkan_error(xc + a / cos_val)
            y = bersihkan_error(yc + b * math.tan(rad))
            points.append((s, x, y))
    return points
```

Gambar 3.4 Kode Hiperbola

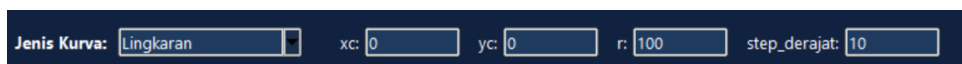
Logika looping pada fungsi `compute_hiperbola` juga membangkitkan himpunan sudut dari 0° sampai dengan 360° sebagaimana pada lingkaran dan elips.

Akan tetapi, sebelum koordinat x dihitung, program terlebih dahulu melakukan pemeriksaan terhadap nilai $\cos(\theta)$ melalui kondisi `if abs(cos_val) > 1e-10`. Pemeriksaan tersebut dilakukan dengan tujuan untuk mencegah terjadinya pembagian dengan nol (division by zero) yang dapat terjadi pada sudut 90° dan 270° , mengingat pada sudut tersebut nilai cosinus mendekati nol sehingga hasil pembagian $a/\cos(\theta)$ akan menuju nilai tak terhingga (mendekati asimtot hiperbola). Apabila kondisi tersebut terpenuhi, maka titik baru akan dihitung dan ditambahkan ke dalam list points. Sebaliknya, apabila kondisi tidak terpenuhi, titik tersebut akan dilewati agar program tidak mengalami kegagalan proses.

3.3 Desain Antarmuka Interaktif

Antarmuka program dirancang dengan tujuan agar pengguna dapat mengubah nilai parameter kurva secara interaktif tanpa harus melakukan perubahan pada kode program secara langsung. Terdapat dua jenis fitur masukkan (input) yang disediakan dalam program ini, yaitu slider dan keyboard (*entry box*), yang masing-masing digunakan untuk mengubah nilai variabel konstanta pada setiap jenis kurva.

a. Input Keyboard (*Entry Box*)

The image shows a dark-themed user interface for inputting curve parameters. It features a dropdown menu labeled 'Jenis Kurva:' with 'Lingkaran' selected. To the right are four text input fields: 'xc:' with '0', 'yc:' with '0', 'r:' with '100', and 'step_derajat:' with '10'.

Gambar 3.5 Input Keyboard

Input keyboard berupa teks (*entry box*) digunakan untuk memasukkan nilai parameter secara manual dengan tingkat presisi yang tinggi, seperti nilai koordinat pusat (x_c , y_c) jari-jari (r), semi-sumbu (a , b), maupun rentang parameter t_1 dan t_2 pada masing-masing kurva. Fitur ini digunakan apabila pengguna bermaksud menguji nilai parameter tertentu secara spesifik, misalnya untuk membandingkan hasil rendering pada nilai $r = 100$ dengan $r = 50$.

Nilai yang dimasukkan melalui *entry box* akan diambil dengan menggunakan fungsi `self.get_val` (“nama parameter”) pada saat tombol perhitungan/render ditekan oleh pengguna. Apabila nilai yang dimasukkan tidak valid, misalnya `step` bernilai nol atau negatif, maka program akan menampilkan kesalahan melalui fungsi `messagebox.showerror()` sehingga pengguna dapat memperbaiki masukan tersebut sebelum proses penggambaran kurva dilakukan kembali.

b. Input Slider



Gambar 3.6 Input Slider

Slider digunakan untuk mengubah nilai parameter secara dinamis dan langsung (real-time) melalui penggeseran penanda pada rentang nilai tertentu. Fitur ini bermanfaat untuk mengamati perubahan bentuk kurva secara visual dan kontinu, misal pada saat slider step bergeser dari 5° menuju 45° , perubahan bentuk kurva dari bentuk yang halus menjadi bentuk poligon dapat diamati secara langsung.

Setiap perubahan posisi slider akan memicu event callback yang secara otomatis memanggil kembali fungsi `compute_<jenis_kurva>()` dan `plot_curve()`, sehingga grafik pada canvas matplotlib akan langsung diperbarui tanpa diperlukan penekanan tombol tambahan. Mekanisme tersebut memberikan pengalaman eksplorasi parameter yang lebih interaktif dibandingkan dengan penggunaan input keyboard saja.

c. Hubungan Antarmuka dengan Variabel Konstanta

Setiap perubahan yang dilakukan baik melalui slider maupun keyboard akan secara langsung memengaruhi nilai variabel konstanta yang digunakan dalam fungsi `compute_lingkaran`, `compute_elips`, `compute_parabola`, dan `compute_hiperbola`. Berikut tabel rangkum parameter yang dapat diatur melalui antarmuka untuk masing-masing jenis kurva.

Tabel 3.3 Hubungan Antarmuka

Jenis Kurva	Parameter yang Diatur	Jenis Input
Lingkaran	xc, yc, r, step derajat	Keyboard & Slider
Elips	xc, yc, a, b, step derajat	Keyboard & Slider
Parabola	xp, yp, a, t1, t2, step	Keyboard & Slider

Hiperbola	xc, yc, a, b, step derajat	Keyboard & Slider
-----------	----------------------------	-------------------

Dengan adanya kombinasi input keyboard dan slider, antarmuka program menjadi lebih fleksibel, yaitu input keyboard memberikan presisi nilai yang tepat, sedangkan slider memberikan kemudahan dalam melakukan eksplorasi visual secara tepat. Setelah nilai parameter diperbarui, program akan menjalankan kembali fungsi `compute_<jenis_kurva>()` untuk menghitung ulang seluruh titik koordinat, kemudian fungsi `plot_curve()` akan menggambar ulang kurva pada canvas dan menampilkan hasilnya melalui fungsi `self.canvas.draw()`.

BAB 4

PEMBAHASAN

4.1 Analisis Perbandingan (Eksplorasi Variasi Nilai Delta (Step))

Pada Bab kali ini, kami akan memberikan penyajian hasil eksplorasi variasi nilai step ($\delta/\Delta t$) pada setiap kurva konik parametrik yang telah diimplementasikan dalam program. Variasi step dilakukan secara sistematis untuk mengamati dampaknya terhadap kerapatan titik yang di render serta kualitas visual kurva yang dihasilkan. Secara umum, terdapat hubungan berbanding terbalik antara nilai step dan jumlah titik yang dihitung, yang secara langsung mempengaruhi kehalusan kurva pada layar.

4.1.1 Variasi Step pada Kurva Lingkaran

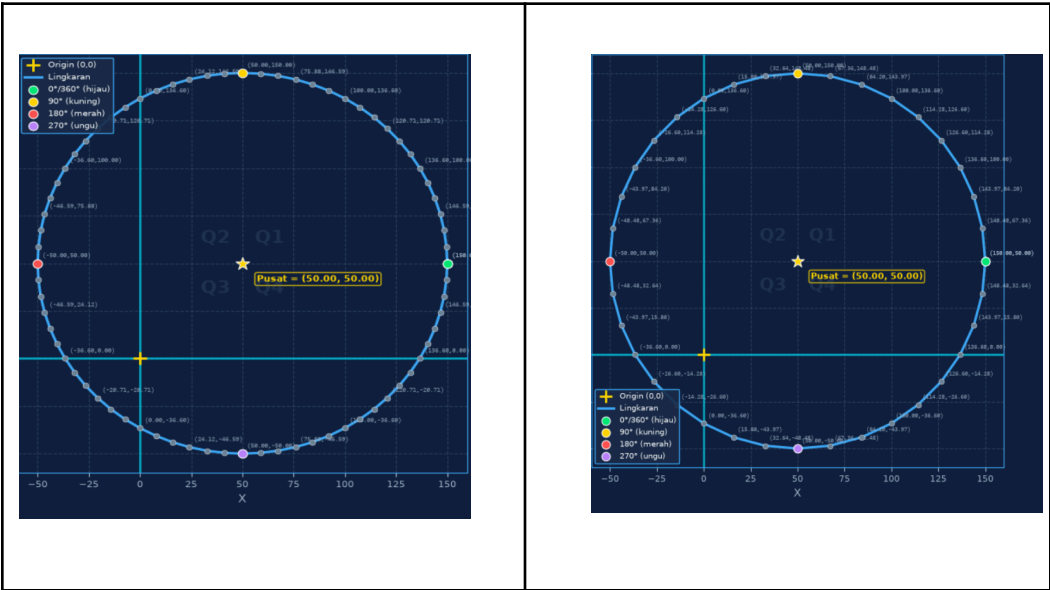
Pada kurva lingkaran, step bekerja secara merata di seluruh keliling karena lingkaran hanya memiliki satu semi-sumbu tunggal yakni jari-jari (r). Titik-titik tersebar dengan jarak busur yang seragam di setiap bagian kurva. Sehingga kekasaran mulai menyerupai poligon beraturan seiring bertambahnya nilai step.

Tabel 4.1 Hasil Eksplorasi Variasi Step pada Kurva Lingkaran

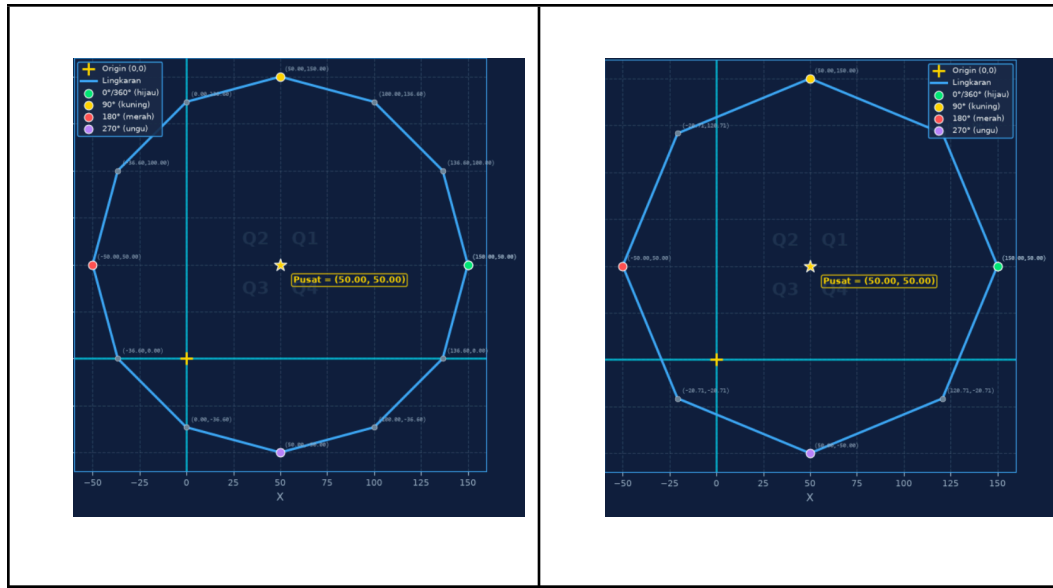
Step (θ)	Jumlah Titik	Kualitas Visual	Beban Komputasi
5°	73 titik	Halus	Tinggi
10°	37 titik	Sedang	Sedang
30°	13 titik	Kasar	Rendah

45°	9 titik	Sangat Kasar	Sangat Rendah
-----	---------	--------------	---------------

Berdasarkan tabel, nilai step 5° - 10° direkomendasikan untuk kurva lingkaran. Pada step besar ($\geq 30^\circ$), berkurangnya jumlah titik menyebabkan kurva yang terbentuk menyerupai poligon beraturan dan kehilangan karakteristik lingkaran yang mulus, sehingga nilai jari-jari (r) tidak lagi tergambar dengan akurat secara visual.



Gambar 4.1 Lingkaran dengan Step 5° dan 10°



Gambar 4.2 Lingkaran dengan Step 30° dan 45°

4.1.2 Variasi Step pada Kurva Elips

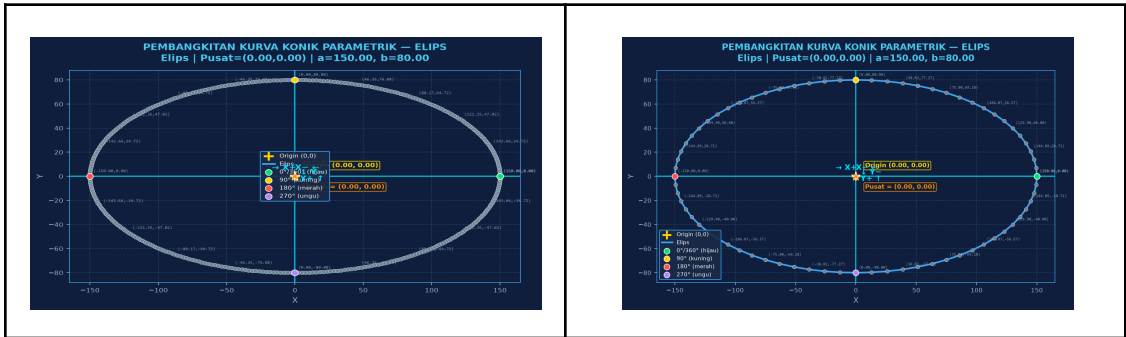
Pada Kurva Elips, step bekerja identik hampir sama dengan lingkaran, namun dampaknya lebih terlihat karena adanya dua semi-sumbu yang berbeda ($a \neq b$). Titik-titik lebih rapat di area ujung sumbu pendek dan lebih renggang di area ujung sumbu panjang. Sehingga kekasaran visual tampak tidak merata ketika step diperbesar.

Tabel 4.2 Hasil Eksplorasi Variasi Step pada Kurva Elips

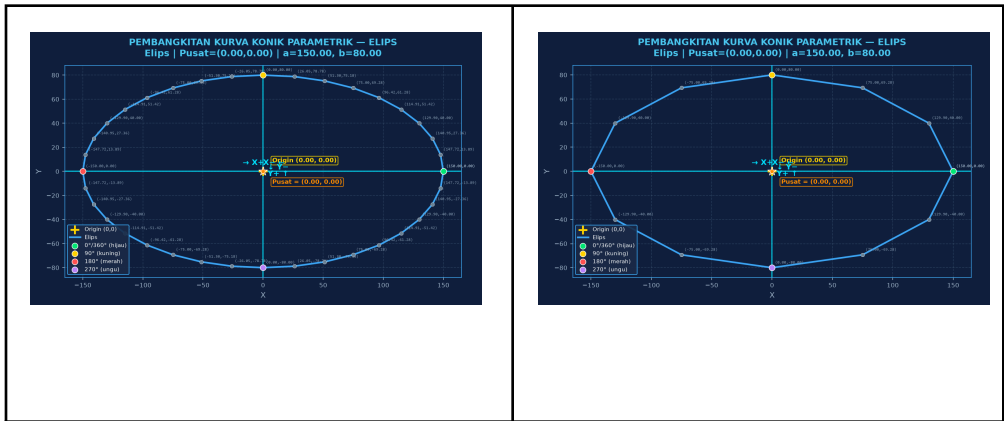
Step (θ)	Jumlah Titik	Kualitas Visual	Beban Komputasi
1°	361 titik	Sangat Halus	Sangat Tinggi
5°	73 titik	Halus	Tinggi
10°	37 titik	Agak kasar	Sedang

30°	13 titik	Bentuk poligon oval	Rendah
45°	8 titik	Sangat kasar	Sangat Rendah

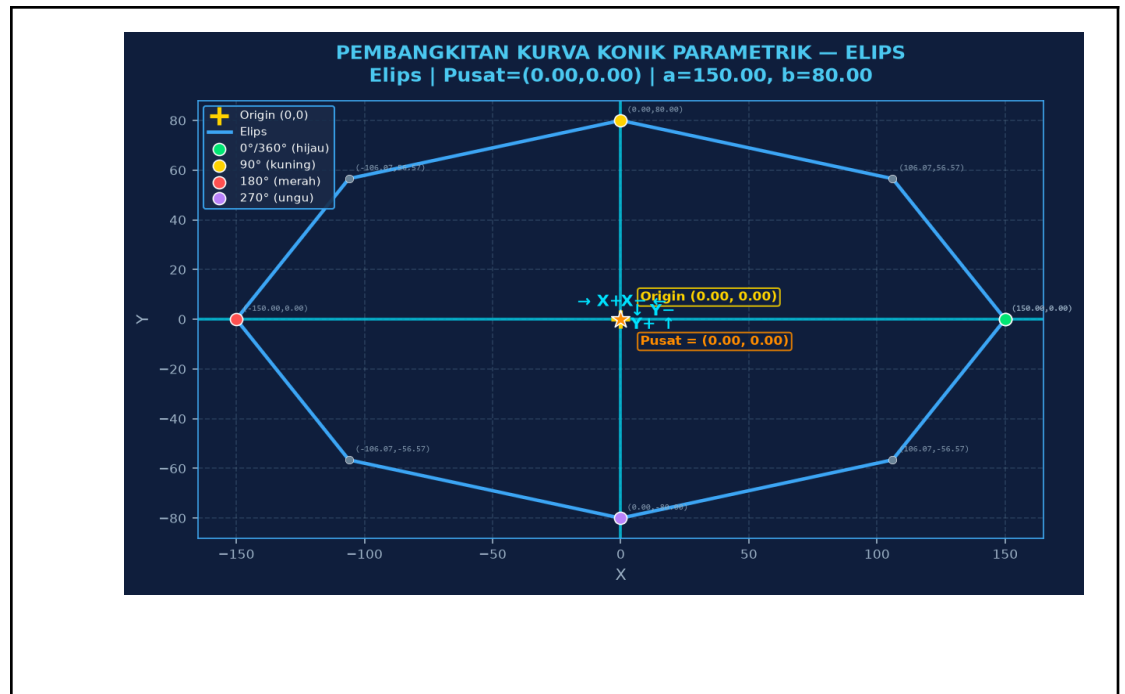
Nilai Step 5° - 10° direkomendasikan untuk elips. Pada step ($\geq 30^\circ$), asimetri antara sumbu horizontal dan vertikal menyebabkan poligon yang terbentuk terlihat tidak beraturan, sehingga hubungan antara parameter a dan b tidak lagi tergambar dengan akurat.



Gambar 4.3 Elips dengan Step 1° dan 5°



Gambar 4.4 Elips dengan Step 10° dan 30°



Gambar 4.5 Elips dengan Step 45°

4.1.3 Variasi Step pada Kurva Parabola

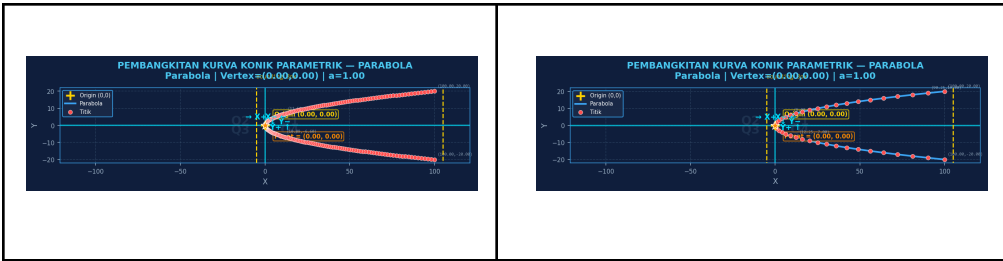
Pada Kurva parabola, parameter step adalah selisih nilai t antar iterasi (Δt). Berbeda dengan lingkaran dan elips, t tidak periodik melainkan bertambah liner dari t_1 hingga t_2 . Oleh karena itu, satuan step bukan derajat melainkan nilai numerik parameter t .

Tabel 4.3 Hasil Eksplorasi Variasi Step pada Kurva Parabola ($X_p = Y_p = 0$, $a = 1$, t : -10 s.d 10)

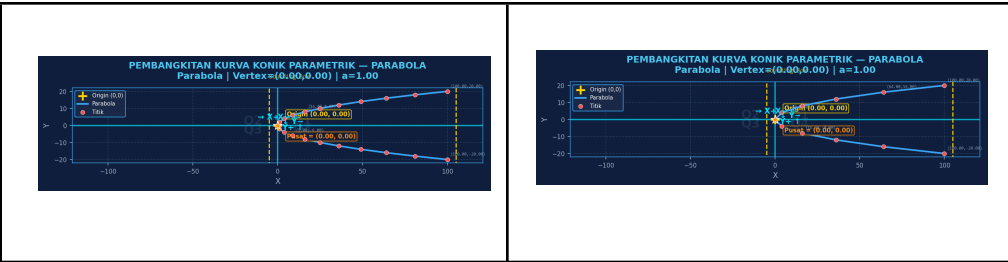
Step (t)	Jumlah Titik	Kualitas Visual	Beban Komputasi
0.1	201 titik (t : -10 s.d 10)	Sangat halus, kurva kontinu mulus	Sangat tinggi
0.5	41 titik	Halus, sedikit kekasaran di lekukan	Tinggi

1.0	21 titik	Terlihat segmen garis lurus antar titik	Sedang
2.0	11 titik	Sangat kasar, poligon terbuka, bukan kurva	Rendah
5.0	5 titik	Hanya beberapa titik, bentuk parabola hilang	Sangat rendah

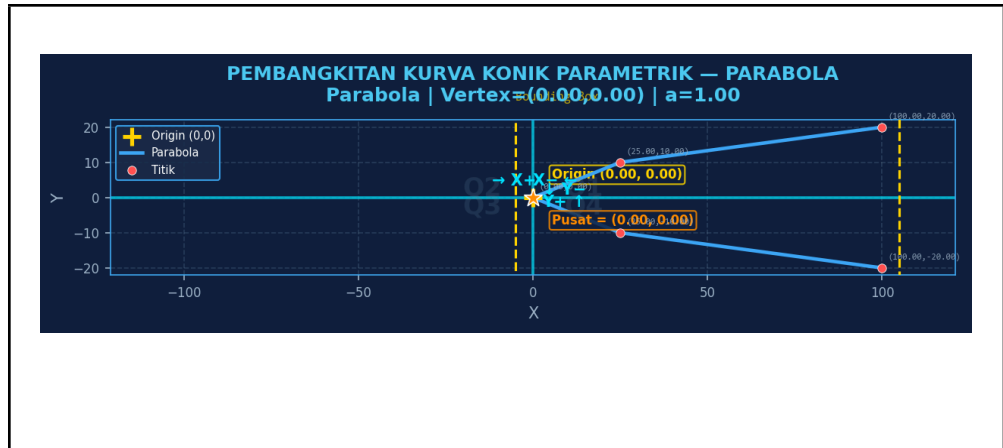
Karena rumus parabola bersifat kuadratik ($x = xp + a \cdot t^2$, $y = yp + 2a \cdot t$), perubahan koordinat lebih drastis di ujung rentang t. Step $\Delta t = 0.5 - 1.0$ menghasilkan kurva yang cukup halus tanpa memakan terlalu banyak iterasi.



Gambar 4.6 Parabola dengan Step (t) 0,1 dan 0,5



Gambar 4.7 Parabola dengan Step (t) 2,0 dan 1,0



Gambar 4.8 Parabola dengan Step (t) 5,0

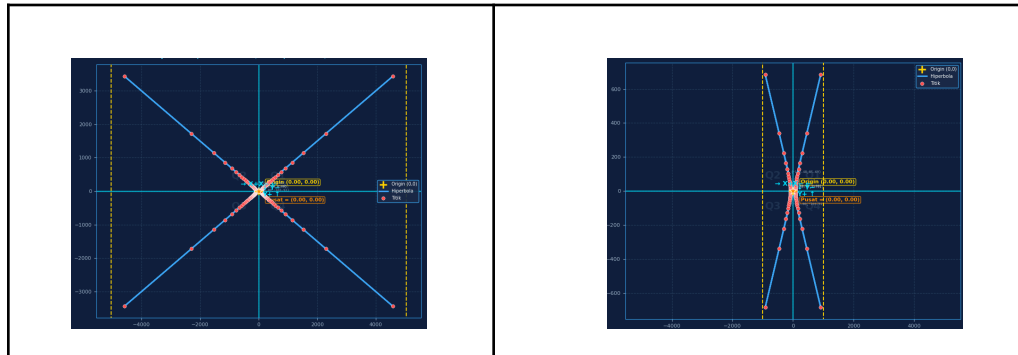
4.1.4 Variasi Step pada Kurva Hiperbola

Pada Kurva Hiperbola, nilai step Δt menentukan jarak antar titik yang dihitung pada proses pembangkitan kurva parametrik. Hiperbola menggunakan fungsi $\sec(\theta)$ dan $\tan(\theta)$ yang bersifat divergen mendekati asimtot, sehingga dampak perubahan step jauh lebih signifikan dibandingkan kurva lainnya.

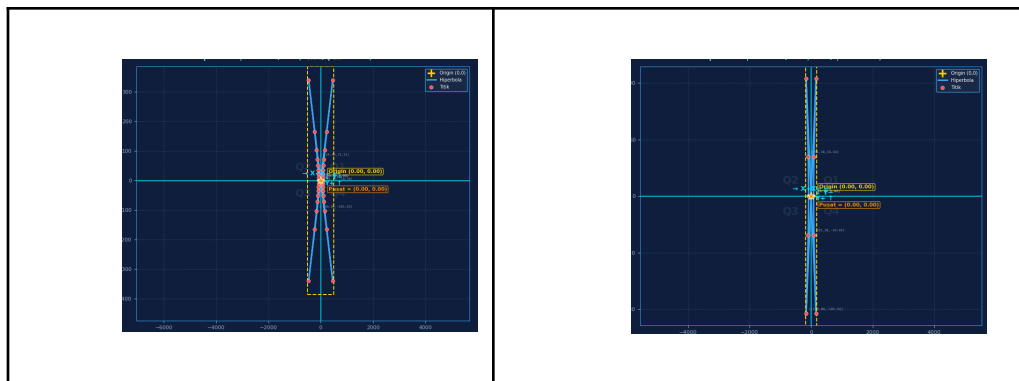
Tabel 4.4 Hasil Eksplorasi Variasi Step pada Kurva Hiperbola

Step (θ)	Jumlah Titik	Kualitas Visual	Beban Komputasi
1°	359 titik	Sangat Halus	Sangat Tinggi
5°	71 titik	Halus	Tinggi
10°	35 titik	Sedang	Sedang
30°	7 titik	Kasar	Sangat Rendah

Semakin kecil step, jumlah titik akan semakin banyak dan kedua cabang hiperbola akan terlihat semakin mulus, terutama di sekitar vertex. Sebaliknya, semakin besar step, jumlah titik akan berkurang dan cabang hiperbola akan tampak seperti ‘patah-patah’ menyerupai garis lurus bersudut tajam, terutama di bagian yang mendekati asimtot. Step $5^\circ - 10^\circ$ merupakan titik seimbang yang direkomendasikan.



Gambar 4.9 Hiperbola dengan Step 1° dan 5°



Gambar 4.10 Hiperbola dengan Step 10° dan 30°

4.2 Analisis Karakteristik Kurva Berdasarkan Teori Geometri

Perubahan karakteristik visual parametrik akibat variasi step dapat dijelaskan secara matematis menggunakan prinsip aproksimasi poligon (polygon approximation) terhadap kurva kontinu.

4.2.1 Mengapa Kurva menjadi poligon saat step diperbesar?

Secara teori, kurva parametrik merupakan himpunan titik kontinu tak hingga. Namun, dalam implementasi komputer, titik-titik tersebut dihasilkan secara diskrit dengan jumlah yang bergantung pada nilai step. Program kemudian menghubungkan titik-titik dengan garis lurus (line segment), bukan kurva melengkung.

Ketika step diperbesar, jarak antar titik yang dihitung menjadi lebih besar. Garis lurus yang menghubungkan dua titik yang berjauhan tidak lagi mengikuti kelengkungan asli kurva, sehingga secara visual kurva tampak bersudut (poligon). Fenomena ini dikenal sebagai Chor Error atau Aproksimasi Poligon.

4.2.2 Analisis Karakteristik per jenis Kurva

a. Lingkaran dan Elips

Untuk lingkaran dengan $r = 100$ dan $\text{step} = 10^\circ$, panjang chord (tali busur) antara dua titik berturut-turut dapat dihitung dengan rumus.

$$\text{Chord} = 2r \sin(\theta/2) = 2(100) \sin(10^\circ/2) = 2(100)(0,0868) = 17,63 \text{ piksel}$$

Semakin besar step, semakin panjang chord, dan kurva semakin terlihat sebagai poligon. Sedangkan pada elips, panjang chord berbeda di setiap titik karena $a \neq b$. Di area sumbu panjang ($a = 150$), chord lebih panjang dibandingkan di area sumbu pendek ($b = 80$). Hal ini menyebabkan kekasaran visual yang tidak merata sesuai dengan step yang sama.

b. Parabola

Parabola bersifat kudaratik: $(x = xp + a \cdot t^2, y = yp + 2a \cdot t)$. Jarak antar titik bertambah secara eksponensial seiring meningkatnya $|t|$. Akibatnya, kurva terlihat lebih kasar di bagian ujung (t jauh dari nol) dibandingkan bagian tengah (t dekat nol), bahkan dengan step yang sama.

c. Hiperbola

Hiperbola berbeda secara fundamental dari lingkaran, elips, dan parabola karena fungsi pembentukannya $\sec(\theta)$ dan $\tan(\theta)$ bersifat divergen (menuju tak

terhingga) saat θ mendekati 90° dan 270° . Akibatnya panjang chord tidak konstan terhadap step dan jauh lebih ekstrem dibanding elips.

Tabel 4.5 Perbandingan Panjang Chord pada Hiperbola ($a=80$, $b=60$, $step=10^\circ$)

Area	Dari titik	Ke titik	Panjang Chord
Dekat vertex ($\theta: 0^\circ \rightarrow 10^\circ$)	(80,00 ; 0,00)	(81,23 ; 10,58)	$\approx 10,65$ piksel
Dekat asimtot ($\theta: 70^\circ \rightarrow 80^\circ$)	(233,90 ; 164,85)	(460,70 ; 340,28)	$\approx 286,73$ piksel

4.3 Analisis Trade-off antara Kualitas Visual dan Performa Komputasi

Analisis trade-off dilakukan untuk memahami hubungan antara kehalusan kurva yang dihasilkan dengan beban komputasi yang diperlukan program. Pemahaman ini penting sebagai dasar pemilihan nilai step yang optimal dalam aplikasi grafika komputer.

4.3.1 Trade-off pada Kurva Lingkaran

Pada proses pembangkitan lingkaran parametrik, nilai step memiliki pengaruh langsung terhadap kualitas visual kurva dan beban komputasi yang diperlukan. Parameter step menentukan interval perubahan sudut yang digunakan untuk menghasilkan titik-titik penyusun lingkaran. Ketika nilai step dibuat lebih besar misalnya 30° , jumlah titik yang dihitung menjadi lebih sedikit. Akibatnya waktu komputasi menjadi lebih cepat dan penggunaan sumber daya komputer lebih ringan. Namun kualitas visual kurva menurun karena jarak antar titik semakin renggang sehingga lingkaran terlihat lebih kasar dan kurang halus.

Sebaliknya ketika nilai step diperkecil menjadi 10° atau 5° jumlah titik yang akan dihasilkan semakin banyak. Kemudian, kurva yang sudah terbentuk menjadi lebih halus dan lebih mendekati bentuk lingkaran matematis yang ideal. Akan tetapi, peningkatan jumlah titik menyebabkan proses perhitungan dan rendering memerlukan waktu yang lebih lama jika dibandingkan penggunaan step yang besar. Berdasarkan hasil eksperimen, terdapat trade-off antara kualitas visual dan performa komputasi. Nilai step yang kecil dapat memberikan kualitas visual yang lebih baik karena kurva tampak lebih mulus, akan tetapi memerlukan lebih banyak operasi perhitungan dan rendering. Sedangkan nilai step yang besar meningkatkan efisiensi komputasi namun, mengurangi kualitas visual kurva.

Dengan demikian, pemilihan nilai step ini harus disesuaikan dengan kebutuhan pengguna. Untuk visualisasi yang mengutamakan keakuratan dan juga estetika disarankan untuk menggunakan step yang lebih kecil. Sedangkan untuk pengguna yang membutuhkan kecepatan proses bisa menggunakan step yang lebih besar dapat digunakan meskipun kualitas visual kurva sedikit berkurang.

Tabel 4.6 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Lingkaran

Step (θ)	Jumlah Titik	Kualitas Visual	Beban Komputasi
5°	73 titik	Halus	Tinggi
10°	37 titik	Sedang	Sedang
30°	13 titik	Kasar	Rendah
45°	9 titik	Sangat Kasar	Sangat Rendah

Berdasarkan hasil pengujian pada program, diketahui jika semakin kecil nilai step maka kurva yang dihasilkan akan semakin halus karena jumlah titik pembentuk kurva semakin banyak. Namun, kondisi tersebut meningkatkan beban komputasi karena program harus melakukan lebih banyak perhitungan koordinat dan proses rendering. Sedangkan jika semakin besar nilai step maka jumlah titik yang dihitung menjadi sedikit sehingga proses komputasinya jadi lebih cepat. Akan tetapi kualitas visual kurjanya menurun karena kurva terlihat lebih kasar. Oleh karena itu terdapat trade-off antara kemulusan kurva dan performa komputasi dalam pemilihan nilai step.

4.3.2 Trade-off pada Kurva Elips

Untuk proses pembangkitan elips parametrik, konsep yang diberikan oleh elips hampir sama dengan yang divisualkan pada lingkaran. Titik pusat yang digunakan difungsikan sebagai titik dasar utama. Pada pengujian elips, dilakukan beberapa perbandingan menggunakan nilai step yang berbeda-beda, seperti 30° , 15° , 10° , 5° . Nilai setiap step ini akan menentukan besar perubahan sudut pada setiap iterasi saat program menghitung titik-titik elips. Dengan rumus yang sudah dipaparkan.

Dari hasil uji coba diketahui kalau ketika menggunakan step yang besar misalnya 30° , jumlah titik yang dihitung relatif sedikit sehingga proses komputasi menjadi lebih cepat. Namun, jarak antar titik cukup jauh sehingga bentuk elips yang dihasilkan terlihat kurang halus dan cenderung menyerupai poligon.

Sebaliknya, ketika step diperkecil menjadi 5° atau 1° , program menghitung jauh lebih banyak titik. Akibatnya, kurva elips menjadi lebih halus dan lebih mendekati bentuk matematis yang sebenarnya. Akan tetapi, karena jumlah iterasi yang bertambah, program juga memerlukan lebih banyak perhitungan fungsi sinus dan cosinus serta lebih banyak proses untuk menggambar titik-titik tersebut.

Tabel Trade-off antar visual dan komputasi :

Tabel 4.7 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Elips

Step (θ)	Jumlah Titik	Kualitas Visual	Beban Komputasi
1°	361 titik	Sangat Halus	Sangat Tinggi
5°	73 titik	Halus	Tinggi
10°	37 titik	Agak kasar	Sedang
30°	13 titik	Bentuk poligon oval	Rendah
45°	8 titik	Sangat kasar	Sangat Rendah

SINGKATNYA...!!

Step kecil $\rightarrow$ visual lebih halus, tetapi proses lebih lama karena jumlah titik yang dihitung lebih banyak.

Step besar $\rightarrow$ proses lebih cepat, tetapi visual menjadi kurang halus karena jumlah titik lebih sedikit.

4.3.3 Trade-off pada Kurva Parabola

Pada kurva parabola, terdapat trade-off antara kualitas visual dan performa komputasi yang ditentukan oleh nilai step (Δt). Semakin kecil nilai Δt , semakin rapat titik-titik yang dihasilkan sehingga kurva tampak lebih halus dan mendekati bentuk

matematis sebenarnya. Namun, jumlah iterasi yang diperlukan juga meningkat secara proporsional, sehingga waktu rendering menjadi lebih lama.

Sebaliknya, penggunaan Δt yang lebih besar mengurangi jumlah iterasi dan mempercepat proses komputasi, tetapi kurva dapat terlihat kurang halus karena jarak antar titik menjadi terlalu jauh. Pada rentang parameter $t = -10$ hingga $t = 10$, penggunaan $\Delta t = 0.1$ menghasilkan sekitar 201 titik, sedangkan $\Delta t = 1.0$ hanya menghasilkan sekitar 21 titik.

Tabel 4.8 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Parabola

Step (t)	Jumlah Titik	Kualitas Visual	Beban Komputasi
0.1	201 titik (t : -10 s.d 10)	Sangat halus, kurva kontinu mulus	Sangat tinggi
0.5	41 titik	Halus, sedikit kekasaran di lekukan	Tinggi
1.0	21 titik	Terlihat segmen garis lurus antar titik	Sedang
2.0	11 titik	Sangat kasar, poligon terbuka, bukan kurva	Rendah
5.0	5 titik	Hanya beberapa titik, bentuk	Sangat rendah

		parabola hilang	
--	--	-----------------	--

Hasil pengamatan menunjukkan bahwa penggunaan $\Delta t = 0.5$ hingga 1.0 memberikan keseimbangan terbaik antara kualitas visual dan efisiensi komputasi. Pada rentang tersebut, bentuk parabola sudah terlihat mulus dan kontinu, sementara jumlah iterasi tetap rendah sehingga tidak membebani proses rendering. Penggunaan Δt di bawah 0.5 hanya memberikan peningkatan kualitas visual yang sangat kecil, tetapi meningkatkan jumlah perhitungan secara signifikan.

4.3.4 Trade-off pada Kurva Hiperbola

Nilai step pada hiperbola mempengaruhi langsung jumlah titik yang akan dihitung (tabel F.1) sekaligus kualitas visual hasil rendering. Ketika step dibuat besar (30° – 45°), jumlah titik turun menjadi hanya 7–11 titik sehingga komputasi $\sec(\theta)$ dan $\tan(\theta)$ jauh lebih cepat dan ringan. Namun karena chord di dekat asimtot jauh lebih panjang dari chord di dekat vertex (tabel F.2), kedua cabang hiperbola tampak bersudut tajam dan kehilangan kelengkungan aslinya bahkan terlihat seperti garis patah, bukan kurva mulus.

Sebaliknya, step kecil (1° – 10°) menghasilkan 35–359 titik. Kurva menjadi jauh lebih halus dan mendekati bentuk matematis ideal, dengan transisi mulus dari vertex menuju asimtot. Namun, program harus melakukan jauh lebih banyak operasi trigonometri ($\sec$ dan $\tan$) serta pengecekan pembagian-nol pada setiap iterasi, sehingga beban komputasi meningkat signifikan.

Tabel 4.9 Rekapitulasi Trade-off kualitas Visual dan Komputasi pada Hiperbola

Step (θ)	Jumlah Titik	Kualitas Visual	Beban Komputasi
1°	359 titik	Sangat Halus	Sangat Tinggi
5°	71 titik	Halus	Tinggi
10°	35 titik	Sedang	Sedang
30°	7 titik	Kasar	Sangat Rendah

Step kecil (1°-10°): visual akurat terutama mempresentasikan kelengkungan menuju asimtot, tapi komputasi lebih berat.

Step besar ($\geq 30^\circ$): ringan dan cepat, tapi bentuk asimtotik tidak tergambar dengan baik.

Maka dari itu step 5°-10° merupakan titik seimbang.

4.4 Rangkuman Analisis Perbandingan

Dari seluruh hasil eksplorasi dan analisis yang telah dilakukan, terlihat bahwa setiap parameter pada kurva parametrik memiliki peran masing-masing yang saling melengkapi. Parameter pusat seperti x_c dan y_c (atau x_p , y_p pada parabola) hanya berfungsi memindahkan letak kurva tanpa mengubah bentuknya, sementara parameter r , a , dan b menentukan seberapa besar atau lebar kurva yang terbentuk. Sedangkan step atau delta theta ($\Delta\theta$) memegang peran paling menentukan dalam kualitas akhir kurva yang dirender.

Hal yang menarik dari hasil pengujian adalah munculnya pola yang sama pada keempat jenis kurva, yaitu adanya hubungan berbanding terbalik antara kehalusan visual dan kecepatan komputasi. Kondisi ini wajar terjadi karena secara teori, komputer tidak benar-benar menggambar kurva kontinu, melainkan hanya menghubungkan titik-titik diskrit yang jumlahnya bergantung pada step yang digunakan.

Meski pola dasarnya sama, keempat kurva tidak merespons perubahan step dengan cara yang identik:

- Lingkaran cenderung paling 'toleran' karena jarak antar titiknya seragam di seluruh sisi.
- Elips sedikit lebih sensitif karena adanya perbedaan ukuran sumbu horizontal dan vertikal.
- Parabola terlihat lebih kasar di bagian ujung kurva karena sifat parameternya yang kuadratik.
- Hiperbola menjadi yang paling rentan karena perhitungannya melibatkan fungsi trigonometri yang nilainya melonjak tajam saat mendekati asimtot.

Tabel 4.10 Rekapitulasi Rekomendasi Nilai Step untuk Kurva Parametris

Kurva	Step Rekomendasi	Titik Optimal	Catatan Khusus
Lingkaran	$5^{\circ} - 10^{\circ}$	37 - 73 titik	Titik merata di seluruh keliling
Elips	$5^{\circ} - 10^{\circ}$	37 - 73 titik	Sensitif di area sumbu pendek
Parabola	$\Delta t = 0.5 - 1.0$	21 - 41 titik	Lebih kasar di ujung rentang t
Hiperbola	$5^{\circ} - 10^{\circ}$	35 - 71 titik	Sangat sensitif mendekati asimtot(garis)

Catatan*: trade-off itu ketika mendapatkan satu keuntungan, yang mana harus mengorbankan hal lain. Dalam konteks ini, kualitas visual yang lebih baik diperoleh dengan pengorbanan performa komputasi dan sebaliknya.

BAB 5

KESIMPULAN

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa representasi parametrik merupakan metode yang efektif untuk membangkitkan kurva konik, yaitu lingkaran, elips, parabola, dan hiperbola. Dengan memanfaatkan satu parameter bebas, setiap titik pada kurva dapat dihitung secara berurutan sehingga proses visualisasi menjadi lebih sederhana dan mudah diimplementasikan menggunakan bahasa pemrograman Python. Hasil implementasi juga menunjukkan bahwa setiap jenis kurva memiliki karakteristik yang berbeda sesuai dengan persamaan parametrik dan parameter pembentuknya.

Pengujian terhadap variasi nilai step menunjukkan bahwa parameter ini memiliki pengaruh yang cukup besar terhadap kualitas hasil rendering. Nilai step yang kecil menghasilkan jumlah titik yang lebih banyak sehingga kurva tampak lebih halus dan mendekati bentuk geometrinya. Sebaliknya, ketika nilai step diperbesar, jumlah titik yang dihitung semakin sedikit sehingga kurva mulai terlihat seperti rangkaian garis lurus atau poligon. Kondisi tersebut juga membuat proses komputasi menjadi lebih ringan karena jumlah perhitungan yang dilakukan berkurang.

Selain itu, hasil Pengujian memperlihatkan bahwa setiap kurva memiliki tingkat sensitivitas yang berbeda terhadap perubahan nilai step. Lingkaran cenderung tetap stabil karena distribusi titiknya relatif merata, sedangkan elips mulai menunjukkan ketidakaturan ketika perbedaan panjang kedua sumbunya semakin besar. Pada parabola, perubahan parameter t memengaruhi penyebaran titik sehingga bagian ujung kurva lebih cepat mengalami perubahan dibandingkan bagian puncaknya. Sementara itu, hiperbola menjadi kurva yang paling sensitif karena adanya asimtot yang menyebabkan perubahan koordinat berlangsung sangat cepat ketika mendekati sudut tertentu.

Secara keseluruhan, tujuan dari tugas besar ini telah tercapai, yaitu mengimplementasikan pembangkitan kurva parametrik menggunakan Python, memahami peran setiap parameter terhadap bentuk kurva, serta menganalisis hubungan antara kualitas visual dan beban komputasi melalui variasi nilai step. Melalui proyek ini, dapat dipahami bahwa pemilihan parameter yang tepat tidak hanya mempengaruhi bentuk geometri yang dihasilkan, tetapi juga berperan penting dalam menghasilkan proses rendering yang efisien dan tetap memberikan kualitas visual yang baik.

DAFTAR PUSTAKA

- Das, S., Li, K., Mundhra, A., Wang, M., & Kalogerakis, E. (2021). *Cloud2Curve: Generation and Vectorization of Parametric Sketches*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 7084–7093.
- Karakılıç, İ., Çelik, S., & Oral, M. (2025). *Bézier Curves and Surfaces with the Blending (α, λ, s) -Bernstein Basis*. Symmetry, 17(2), 219.
<https://doi.org/10.3390/sym17020219>
- Eze, dkk(2022). Graphics evolutionary computations in higher order parametric Bezier curves. Computer Systems Science and Engineering, 41(2), 595–609.
<https://doi.org/10.32604/csse.2022.020835>
- Unterguggenberger, dkk. (2024). Fast rendering of parametric objects on modern GPUs. Eurographics Symposium on Parallel Graphics and Visualization.
<https://doi.org/10.2312/pgv.20241129>
- Latour, P., & Van Droogenbroeck, M. (2024). CURDIS: A template for incremental curve discretization algorithms and its application to conics. Virtual Reality & Intelligent Hardware, 6(5), 358-382.
<https://doi.org/10.2312/pgv.20241129>

LAMPIRAN

Berikut adalah kode program python terkait pembangkitan kurva parametris yang kami gunakan pada penugasan ini bisa diakses pada link di bawah ini :

<https://github.com/adeansyafa/Grafika-Komputer.git>